

6. 풀스택 웹 서비스와 API의 이해

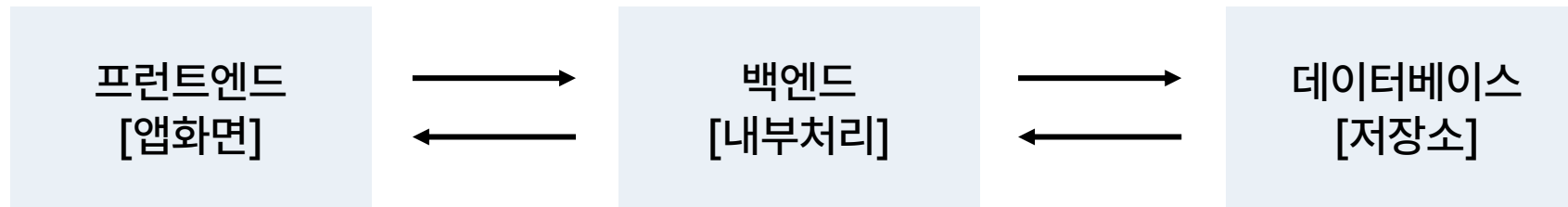


1) 웹/앱 서비스의 구조 이해



웹/앱 서비스의 핵심 요소

- 프론트엔드: 사용자가 보는 모든것
- 백엔드: 보이지 않는 곳에서 일하는 두뇌
- 데이터베이스: 모든 정보의 저장소



프론트엔드(Front-end)

- 웹사이트나 앱에서 사용자가 직접 보고 상호작용하는 화면과 인터페이스를 만드는 영역입니다.
 - 쉽게 말해 집의 인테리어와 외관을 꾸미는 일이며, 버튼, 글자, 이미지, 레이아웃 등을 다룹니다.
- 주요 역할과 특징
 - 화면 구현: 디자이너가 만든 시각적 디자인을 실제 작동하는 웹이나 앱 화면으로 바꿉니다.
 - 사용자 경험(UX) 향상: 사용자가 빠르고 편리하게 서비스를 쓰도록 속도를 높이고 오류를 줄입니다.
 - 서버와 소통: 백엔드 서버에서 데이터를 받아와 화면에 알맞게 보여줍니다.

프론트엔드(Front-end) 관련 기술

- HTML: 웹페이지의 뼈대와 구조를 잡습니다.
- CSS: 색상, 모양, 레이아웃 등 디자인과 스타일을 입힙니다.
- JavaScript / TypeScript: 버튼 클릭이나 움직임 같은 동적 기능을 담당하고 데이터를 처리합니다.
- 프레임워크: 작업 효율을 높이는 React, Vue.js 등을 주로 사용합니다.

백엔드(Back-end)

- 웹사이트나 앱에서 사용자가 볼 수 없는 서버, 데이터베이스, API 등을 구축하고 관리하는 역할입니다.
 - 프론트엔드가 눈에 보이는 '집의 외관'이라면, 백엔드는 보이지 않는 '보일러, 전기 배선, 수도관' 같은 핵심 핵심 인프라를 만드는 일입니다.
- 주요 역할과 특징
 - 데이터 관리: 사용자가 회원가입, 글쓰기, 로그인 등을 할 때 생기는 데이터를 안전하게 저장하고 불러옵니다.
 - 비즈니스 로직 처리: 결제 승인, 상품 추천, 회원 인증처럼 서비스가 돌아가는 실제 규칙과 기능을 처리합니다.
 - 보안 및 안정성: 개인정보를 암호화하고, 해킹 공격을 막으며, 사용자가 몰려도 서버가 터지지 않게 유지합니다

백엔드(Back-end) 관련 기술

- 프로그래밍 언어: 서버를 움직이는 코드를 짜기 위해 Java, Python, Node.js(JavaScript/TypeScript), Go 등을 사용합니다.
- 프레임워크: 개발 속도를 높여주는 Spring Boot(Java), Express(Node.js), Django(Python) 등이 대표적입니다.
- 데이터베이스(DB): 데이터를 보관하는 기술로 MySQL, PostgreSQL(관계형)이나 MongoDB(비관계형)를 씁니다.
- 인프라 및 클라우드: 서버를 배포하고 운영하는 AWS, Docker, Linux 환경에 대한 이해가 필요합니다.

데이터베이스(Database, DB)

- 컴퓨터 시스템에 전자적으로 저장되는 체계적인 데이터의 집합입니다.
- 주요 역할과 특징
 - 데이터 통합 저장: 여러 사람이나 프로그램이 같은 데이터를 함께 쓸 수 있도록 한곳에 모아 관리합니다.
 - 빠른 검색 및 수정: 수백만 건의 데이터 중 사용자가 원하는 정보만 몇 밀리초(ms) 안에 찾아내거나 고칩니다.
 - 데이터 무결성 유지: 데이터가 겹치거나 오류가 생기지 않도록 논리적 규칙을 세워 정확성과 일관성을 지킵니다.
 - 안전한 권한 관리: 아무나 정보를 보거나 지우지 못하게 접근 권한을 제어하고, 문제가 생기면 백업으로 복구합니다.

데이터베이스(Database, DB) 관련 기술

- SQL (구조화 질의어): 관계형 데이터베이스와 소통하기 위해 "데이터 찾아줘", "저장해줘"라고 명령하는 표준 언어입니다.
- 관계형 DB (RDBMS): 데이터를 표(Table) 형태로 정리하고 표끼리 관계를 맺는 기술로, MySQL, PostgreSQL, Oracle 등이 있습니다.
- 비관계형 DB (NoSQL): 정해진 표 없이 자유로운 형태(JSON 문서, 키-값 등)로 대용량 데이터를 다루며, MongoDB, Redis 등이 대표적입니다.
- 설계 및 최적화: 데이터 구조를 짜는 '모델링(정규화)'과 검색 속도를 획기적으로 높이는 '인덱스(Index)' 활용 능력이 필요합니다

2) 바이트코딩과 풀스택 기술의 활용



풀스택개발이란?

- 애플리케이션의 프론트엔드와 백엔드를 모두 개발하는 프로세스
- 핵심 특징 및 역할
 - 프론트엔드(Front-end):
 - 사용자가 직접 마주하는 웹 화면과 인터페이스를 뜻합니다.
 - 버튼, 레이아웃, 시각적 디자인, 입력 폼 등을 구현합니다.
 - 주로 HTML, CSS, JavaScript, React 등을 사용합니다.
 - 백엔드(Back-end):
 - 눈에 보이지 않는 서버측 로직과 데이터 관리 인프라를 뜻합니다.
 - 데이터베이스 관리, API 설계, 사용자 인증 및 인가 처리를 담당합니다.
 - 주로 Java, Spring Boot, Node.js, Python 등을 활용합니다.

풀스택 개발의 장점

- 풀스택은 단순히 화면과 서버를 둘 다 한다는 것을 넘어, 웹 애플리케이션 전체 스택(stack)을 다룰 줄 아는 전문성의 의미합니다.
 - 서비스 전체 구조 이해: 화면부터 서버까지 흐름을 전체적으로 파악하여 효율적이고 견고한 개발 가능.
 - 높은 범용성: 기획부터 배포까지 스스로 진행할 수 있어 소규모 프로젝트나 프로토타입 제작에 매우 유리.
- 주요 기술 스택 예시: MEAN(MongoDB, Express.js, Angular, Node.js), LAMP(Linux, Apache, MySQL, PHP) 등.

풀스택 개발자

- 프론트엔드와 백엔드 기술 스택을 모두 이해하고 스스로 프로젝트 전체를 구축할 수 있는 능력을 갖춘 개발자를 의미합니다.
- 단일 코드베이스 안에서 기획부터 데이터베이스 설계, 서버 구축, 클라이언트 배포까지 전 과정을 통합적으로 처리합니다.
- 인력이 한정된 초기 스타트업이나 프로토타입(MVP)을 빠르게 만들어야 하는 환경에서 선호도가 높습니다.

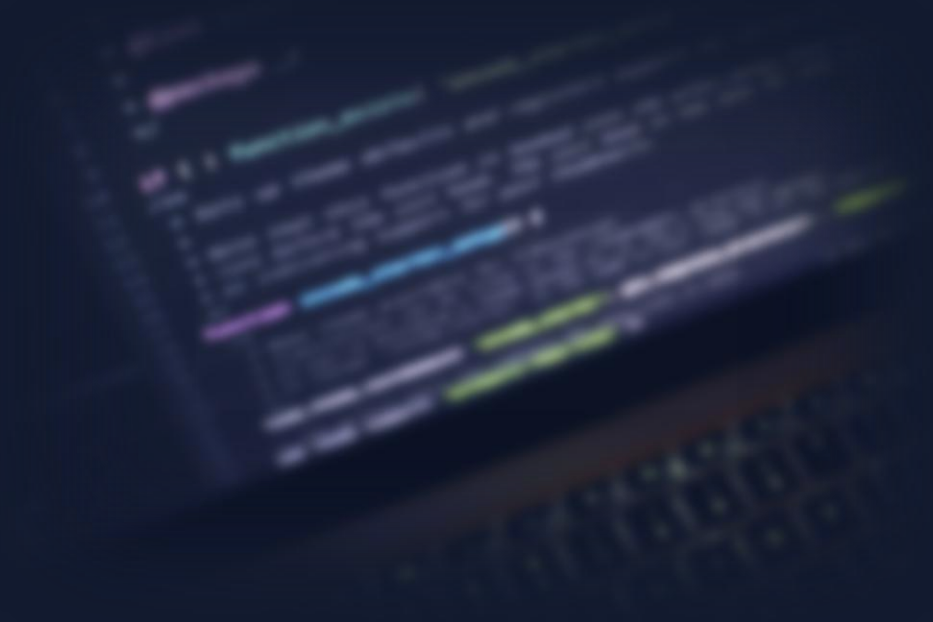
AI 터미널 도구가 가져온 풀스택 개발의 변화

- 바이트 코딩은 "기술을 몰라도 만든다"가 아닌 핵심 개념만 이해하고 나머지는 AI와 협업한다는 접근입니다.
- 서비스는 다음과 같은 역할이 결합되어 완성됩니다.
 - 백엔드 : 데이터처리, 인증 , API
 - 프론트엔드 : 화면, UI, UX
 - 배포와 운영 : 서버, 로그 모니터링

입문자를 위한 AI기반 풀스택 로드맵

- 1) Python : AI가 가장 잘 다루는 언어
- 2) SQL : 서비스 데이터의 기본 구조 이해
- 3) Flask 또는 FastAPI : AI가 만들기 쉬운 백엔드
- 4) HTML, CSS, JavaScript : 화면 구조를 이해하기 위한 기초
- 5) Docker : 배포 자동화 이해를 위한 최소 개념
- 6) Flutter : 앱 UI를 빠르게 만들고 싶을때 선택

3) API

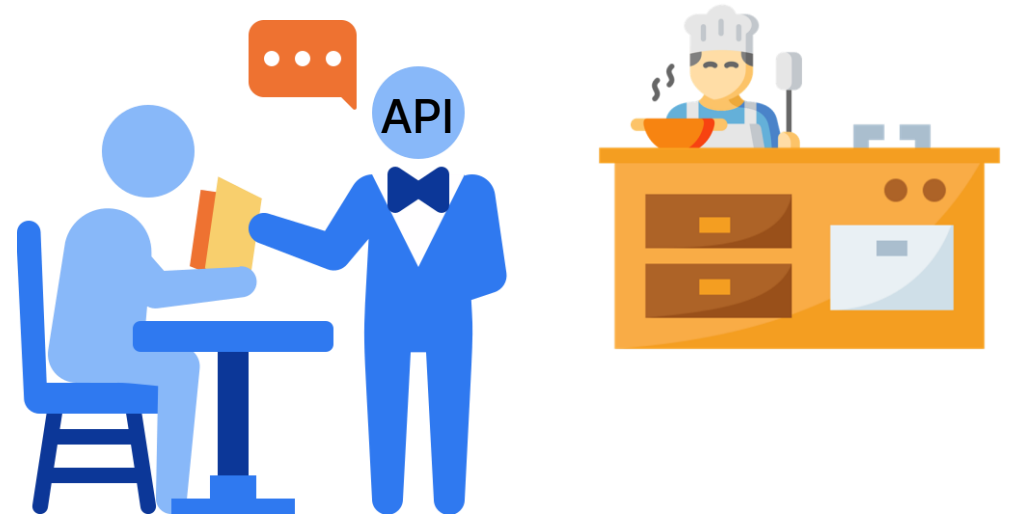


API란?

- API(Application Programming Interface)는 서로 다른 소프트웨어 프로그램들이 데이터를 주고받으며 소통할 수 있도록 연결해 주는 매개체이자 규칙입니다.
- 주요 역할과 특징
 - 데이터 교량: 프론트엔드(화면)와 백엔드(서버/DB)가 데이터를 안전하고 정확하게 주고받도록 중재합니다.
 - 기능의 재사용: 로그인, 결제, 날씨 정보 같은 복잡한 기능을 직접 개발하지 않고, 외부 API를 가져와 내 서비스에 쉽게 붙일 수 있습니다.
 - 보안 및 제어: 내부 데이터베이스를 외부에 직접 노출하지 않고, API라는 허가된 창구를 통해서만 제한적으로 접근하게 만들어 보안을 지킵니다.

API 설정

- 손님이 메뉴판을 보고 음식을 주문하면 주방에 전달하고 완성된 음식을 가져다주는 '식당의 점원(웨이터)'에 비유할 수 있습니다.
 - 복잡한 시스템을 직접 구축하는 것이 아니라, API에게 요청해서 필요한 결과만 추출
 - Gemini CLI를 사용하면 복잡한 연결 코드를 직접 작성할 필요 없이, Gemini API 설정부터 오류 처리까지 모든 과정을 자동으로 처리



API 관련 기술

- REST API: 가장 대중적인 API 아키텍처 스타일로, URL을 통해 자원을 명시하고 HTTP 메서드(GET, POST, PUT, DELETE)로 상태를 전달합니다.
- JSON: API를 통해 데이터를 주고받을 때 가장 널리 쓰이는 텍스트 기반의 가볍고 읽기 쉬운 데이터 형식입니다.
- 인증 및 보안: 허가된 사용자만 API를 쓸 수 있도록 검증하는 기술로 API Key, OAuth 2.0, JWT(Json Web Token) 등을 활용합니다.

API 활용 예시

- API는 이미 일상 곳곳에서 작동하고 있습니다.
- 내가 직접 코드를 짜지 않아도, API 덕분에 서비스들이 연결되어 편리한 경험을 만들어줍니다.
 - 지도 API 배달앱·부동산앱에 구글 지도가 그대로 뜨는 이유. 각 앱이 지도 API를 호출해 화면에 표시합니다.
 - 결제 API 쇼핑몰에서 카카오페이·토스 결제가 되는 구조. 쇼핑몰이 결제사 API를 연동한 결과입니다.
 - 자동화 API Make·Zapier에서 구글 시트 데이터를 슬랙으로 보내는 흐름. 각 서비스의 API를 노코드로 연결한 것입니다.

AI API란? - 서비스에 인공지능 두뇌를 연결하는 방법

- AI 모델을 처음부터 직접 학습·구축하지 않고, 빅테크가 만든 고성능 LLM 및 AI 기능을 내 웹 서비스에 부품처럼 가져다 쓰는 연결 통로입니다.
- 예 :
 - Gemini, Claude, ChatGPT, Gemma 등 AI 개발사들이 API로 자사 모델 제공 요청 시 AI가 직접 응답하는 것처럼 결과 수신

AI API 유형

- LLM/텍스트 생성 API (OpenAI, Anthropic Claude, Google Gemini)
 - 프롬프트와 데이터를 보내면 요약, 번역, 챗봇 답변, 코드 생성 결과를 반환
 - 활용 예: 고객 문의 자동 응답 챗봇, 블로그 글 자동 생성기
- 멀티모달 & 비전 API (Vision, OCR, 음성인식/TTS)
 - 이미지·오디오 데이터를 보내면 이미지 속 텍스트 추출, 사물/상황 인식, 음성 텍스트 변환 수행
 - 활용 예: 영수증 사진 업로드 시 자동 지출 정리, 음성 비서 기능
- 생성형 미디어 API (DALL·E, Midjourney API, ElevenLabs 등)
 - 텍스트 묘사(프롬프트)를 보내면 맞춤형 이미지, 비디오, 감정 실린 음성 합성 반환
 - 활용 예: 사용자 맞춤형 캐릭터 생성, AI 더빙 서비스

AI 모델 API를 사용하는 방법

■ 오픈라우터 같은 API 통합 플랫폼을 사용

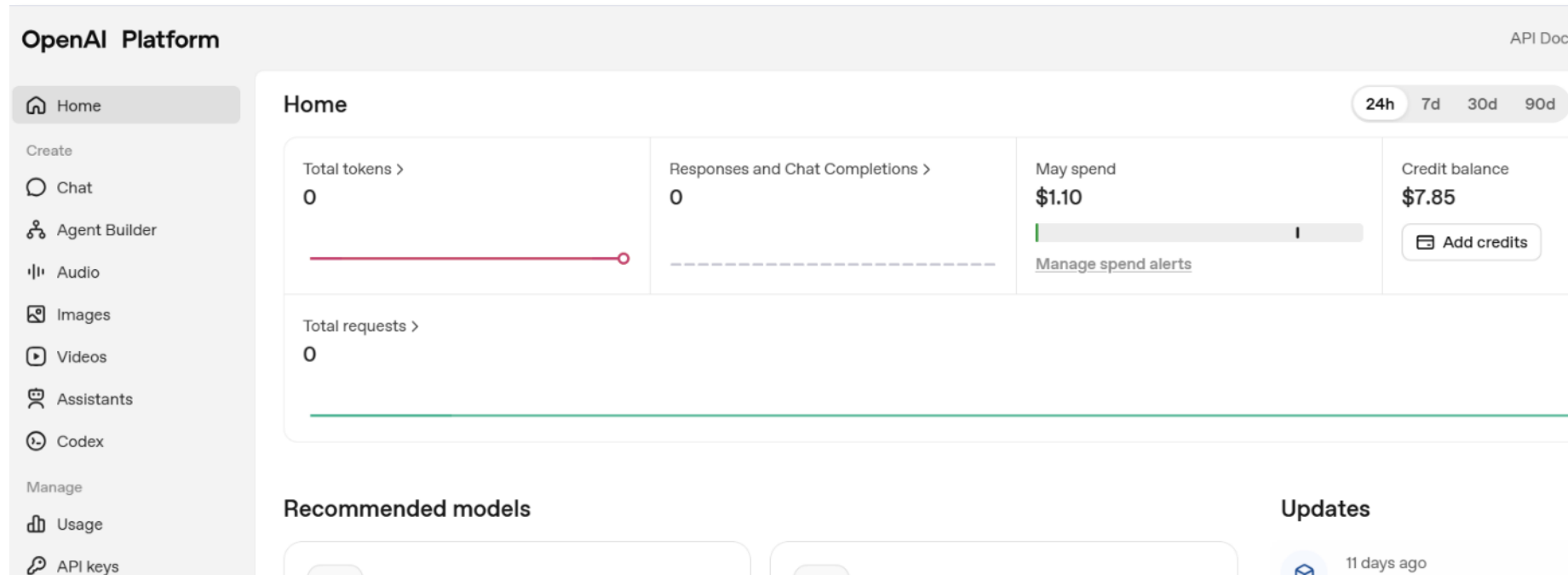
- 마치 푸드코트에서 여러 식당의 음식을 한 곳에서 주문하듯, 오픈라우터에서는 클로드(Claude), GPT, 라마(Llama) 등 다양한 AI 모델을 한곳에서 제공
- API 키를 발급받기만 하면 모든 모델을 자유롭게 전환해 쓸 수 있고, 사용량도 한 화면에서 관리할 수 있어 매우 편리

■ 각 AI 회사의 공식 API를 직접 사용

- 예) 오픈AI에서 GPT API를, 앤트로픽에서 클로드 API를 직접 연동
- 이 경우 회사마다 회원가입을 하고 별도의 API 키를 발급받아야 하므로 관리가 다소 번거롭지만,
- 최신 기능을 가장 빠르게 이용

예시 : OpenAI API

- 유료 서비스인 ChatGPT Plus 구독과는 다른 OpenAI에서 제공하는 GPT 모델을 API로 호출하여 사용하는 방법
 - 프로그래밍을 통해 메시지를 서버로 보내면 응답을 받는 형태로, 다양한 애플리케이션 구현이 가능함



API를 활용한 애플리케이션 만들기

- 마치 전화로 전문가에게 도움을 요청하듯, 프로그램이 AI 모델에게 사진을 분석해 달라고 요청하면 AI가 이미지를 해석하고 결과를 돌려줍니다.
- 애플리케이션은 해당 정보를 화면에 표시하거나, '이 재료로 레시피를 만들어 줘'와 같은 추가 요청을 AI 모델 API에 전달해 결과를 받아볼 수도 있습니다.
- 웹 애플리케이션은 사용자가 사진을 업로드할 인터페이스만 제공하고, 이미지 인식과 텍스트 생성 같은 핵심 기능은 AI 모델 API가 작업합니다.

AI API 연동 아키텍처 : 프론트엔드와 백엔드의 데이터 흐름

- 사용자의 요청이 AI 모델을 거쳐 웹 화면에 표시되기까지의 4단계 전체 동작 원리입니다.
 - 1단계: 사용자 입력 (프론트엔드) 사용자가 웹 화면의 입력창에 텍스트를 입력하거나 이미지 업로드 후 전송 버튼 클릭
 - 2단계: 백엔드 서버로 요청 전달 (프론트 → 백엔드) 프론트엔드는 AI API를 직접 부르지 않고, 자체 백엔드 서버의 엔드포인트(예: /api/chat)로 사용자 데이터를 전송
 - 3단계: 안전한 AI API 호출 (백엔드 → AI 서비스) 백엔드 서버가 환경변수(.env)에 숨겨둔 API 키와 시스템 프롬프트를 조합하여 외부 AI API로 요청 전송
 - 4단계: 응답 가공 및 화면 출력 (AI → 백엔드 → 프론트엔드) AI가 생성한 응답 텍스트(또는 스트리밍 조각)를 받아 브라우저로 전달하고 웹 화면에 실시간 표시

AI API를 안전하고 효율적으로 다루기 위한 가이드

- API 키(Key) 발급과 보안 관리
 - 서비스 식별 및 과금을 위한 '비밀번호' 역할
 - 주의: 웹 브라우저(프론트엔드 코드)에 키를 노출하지 않고, 반드시 백엔드 환경변수(.env)로 안전하게 관리
- 토큰(Token)과 비용 최적화
 - 글자/단어 단위의 사용량 기준 과금(Input/Output 토큰)
 - 프롬프트를 간결하게 유지하고, 불필요한 반복 호출을 줄여 비용 절감



THANK YOU!